

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	9
2 МЕТОД РЕШЕНИЯ.....	11
3 ОПИСАНИЕ АЛГОРИТМОВ.....	15
3.1 Алгоритм конструктора класса cl_base.....	15
3.2 Алгоритм деструктора класса cl_base.....	16
3.3 Алгоритм метода set_name класса cl_base.....	16
3.4 Алгоритм метода get_name класса cl_base.....	17
3.5 Алгоритм метода get_head класса cl_base.....	18
3.6 Алгоритм метода get_subordinate_count класса cl_base.....	18
3.7 Алгоритм метода get_subobject_by_name класса cl_base.....	18
3.8 Алгоритм метода get_subobject_by_index класса cl_base.....	19
3.9 Алгоритм метода print_tree класса cl_base.....	20
3.10 Алгоритм конструктора класса cl_application.....	21
3.11 Алгоритм метода build_tree_objects класса cl_application.....	21
3.12 Алгоритм метода exec_app класса cl_application.....	23
3.13 Алгоритм функции main.....	24
3.14 Алгоритм конструктора класса cl_1.....	25
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	26
5 КОД ПРОГРАММЫ.....	39
5.1 Файл cl_1.cpp.....	39
5.2 Файл cl_1.h.....	39
5.3 Файл cl_application.cpp.....	39
5.4 Файл cl_application.h.....	42
5.5 Файл cl_base.cpp.....	42

5.6 Файл cl_base.h.....	45
5.7 Файл main.cpp.....	46
6 ТЕСТИРОВАНИЕ.....	47
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	48

1 ПОСТАНОВКА ЗАДАЧИ

Для организации иерархического построения объектов необходимо разработать базовый класс, который содержит функционал и свойства для построения иерархии объектов. В последующем, в приложениях использовать этот класс как базовый для всех создаваемых классов. Это позволит включать любой объект в состав дерева иерархии объектов.

Каждый объект на дереве иерархии имеет свое место и наименование. Не допускается для одного головного объекта одинаковые наименования в составе подчиненных объектов.

Создать базовый класс со следующими элементами:

- свойства:
 - наименование объекта (строкового типа);
 - адрес головного объекта для текущего объекта (для корневого объекта значение адреса равен nullptr);
 - динамический массив адресов объектов, подчиненных к текущему объекту в дереве иерархии.
- функционал:
 - параметризированный конструктор с параметрами: указатель на объект базового класса, содержащий адрес головного объекта в дереве иерархии; строкового типа, содержащий наименование создаваемого объекта (имеет значение по умолчанию);
 - метод редактирования имени объекта. Один параметр строкового типа, содержит новое наименование объекта. Если нет дуближа имени подчиненных объектов у головного, то редактирует имя и возвращает «истину», иначе возвращает «ложь»;
 - метод получения имени объекта;

- о метод получения адреса головного объекта текущего объекта;
- о метод получения количества подчиненных объектов текущего объекта;
- о метод вывода наименований объектов в дереве иерархии слева направо и сверху вниз;
- о метод получения адреса непосредственно подчиненного объекта по его имени. Если объект не найден, то возвращает nullptr. Один параметр строкового типа, содержит наименование искомого подчиненного объекта;
- о метод получения адреса непосредственно подчиненного объект по его индексу. Допустимые значения индекса от 1 до количества подчиненных объектов. Индексация подчиненных объектов выстраивается согласно порядку добавления головному объекту. Если значение индекса не принадлежит данному интервалу, то возвращает nullptr. У метода один параметр целого типа, содержит значение индекса подчиненного объекта;
- о метод деструктора, который удаляет все объекты на ветке текущего объекта..

Для построения дерева иерархии объектов в качестве корневого объекта используется объект приложение. Класс объекта приложения наследуется от базового класса. Объект приложение реализует следующий функционал:

- метод построения исходного дерева иерархии объектов (конструирования моделируемой системы);
- метод запуска приложения (начало функционирования системы, выполнение алгоритма решения задачи).

Написать программу, которая последовательно строит дерево иерархии объектов, слева направо и сверху вниз. Переход на новый уровень происходит

только от правого (последнего) объекта предыдущего уровня. Для построения дерева использовать **объекты двух производных классов**, наследуемых от базового. Исключить создание объекта если его наименование совпадает с именем уже имеющегося подчиненного объекта у предполагаемого головного. Исключить добавление нового объекта, не последнему подчиненному предыдущего уровня.

Построчно, по уровням вывести наименования объектов построенного иерархического дерева.

Поменять наименования объектов на уровнях иерархии выбранных по их индексам. Если номер уровня или индекса заданы некорректно, то редактирование не выполняется. После редактирования имен построчно, по уровням вывести наименования объектов иерархического дерева.

Основная функция должна иметь следующий вид:

```
int main ( )
{
    cl_application  ob_cl_application ( nullptr ); // создание корневого
    объекта
    ob_cl_application.build_tree_objects ( );      // конструирование
    системы, построение дерева объектов
    return ob_cl_application.exec_app ( );        // запуск системы
}
```

Наименование класса `cl_application` и идентификатора корневого объекта `ob_cl_application` могут быть изменены разработчиком.

Все версии курсовой работы имеют такую основную функцию.

В данной версии курсовой работы в методе `build_tree_objects ( )` выполняется только построение дерева иерархии объектов.

1.1 Описание входных данных

Первая строка:

«имя корневого объекта»

Вторая строка и последующие строки:

«имя головного объекта» «имя подчиненного объекта»

Создается подчиненный объект и добавляется в иерархическое дерево. Если «имя головного объекта» равняется «имени подчиненного объекта», то новый объект не создается и построение дерева объектов завершается.

Следующая строка и последующие строки

«номер уровня» «индекс элемента на уровне» «новое наименование объекта»

Если «номер уровня» равно нулю, то редактирование имен объектов завершается.

Для изменения наименования корневого объекта строка входных данных имеет вид:

1 1 «новое наименование корневого объекта»

Пример ввода:

```
Object_root
Object_root Object_1
Object_root Object_2
Object_root Object_3
Object_3 Object_4
Object_3 Object_5
Object_6 Object_6
1 1 root_Object
3 1 Object_15
4 1 Object_41
2 7 Object_72
2 2 Object_55
0
```

Дерево объектов, которое будет построено по данному примеру:

```
Object_root
  Object_1
  Object_2
  Object_3
    Object_4
    Object_5
```

1.2 Описание выходных данных

Первая строка:

«имя корневого объекта»

Вторая строка и последующие строки имена головного и подчиненных объектов очередного уровня разделенных двумя пробелами.

«имя головного объекта» «имя подчиненного объекта»[[«имя подчиненного объекта»]]

Со следующей строки аналогичный вывод с отредактированными наименованиями объектов.

Пример вывода:

```
Object_root
Object_root Object_1 Object_2 Object_3
Object_3 Object_4 Object_5
root_Object
root_Object Object_1 Object_55 Object_3
Object_3 Object_15 Object_5
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `ob_cl_application` класса `cl_application` предназначен для создания корневого объекта приложения, управления построением дерева иерархии и запуском системы;
- объект подчинённый класса `cl_1` предназначен для построения дерева иерархии объектов в количестве согласно вводу;
- функция `main` для организации работы программы: создания объекта приложения, вызова методов построения дерева и запуска системы;
- `vector` - массив для динамического хранения указателей на подчинённые объекты;
- `iostream` для управлением потока ввода/вывода;
- `string` для работы с последовательностями символов;
- операторы для динамического выделения памяти `new` и её освобождения `delete`.

Класс `cl_base`:

- свойства/поля:
 - о поле хранение наименования объекта:
 - наименование — `s_object_name`;
 - тип — строка;
 - модификатор доступа — `protected`;
 - о поле хранение адреса на головной объект:
 - наименование — `p_head_object`;
 - тип — указатель на объект базового класса `cl_base`;
 - модификатор доступа — `protected`;
 - о поле динамический массив для хранения адресов

подчинённых объектов:

- наименование — subordinate_objects;
 - тип — вектор указателей на объекты базового класса cl_base;
 - модификатор доступа — protected;
- функционал:
 - метод cl_base — параметризованный конструктор;
 - метод ~cl_base — деструктор для удаления текущего объекта и всех его подчинённых объектов;
 - метод set_name — редактирует имя объекта с проверкой отсутствия дуближа среди подчинённых объектов у головного;
 - метод get_name — возвращает наименование объекта;
 - метод get_head — возвращает адрес головного объекта текущего объекта;
 - метод get_subordinate_count — возвращает количество подчинённых объектов текущего объекта;
 - метод get_subobject_by_name — возвращает адрес подчинённого объекта по его имени (если не найден - возвращает nullptr);
 - метод get_subobject_by_index — возвращает адрес подчинённого объекта по его индексу (от 1 до количества подчинённых);
 - метод print_tree — выводит наименования всех объектов дерева иерархии слева направо и сверху вниз.

Класс cl_application:

- функционал:
 - метод cl_application — параметризованный конструктор;

- о метод `build_tree_objects` — строит исходное дерево иерархии объектов: чтение входных данных, создание объектов с проверкой уникальности имён и правил добавления;

- о метод `exes_app` — запускает систему: чтение команд редактирования имён объектов по уровням и индексам, изменение имён и вывод обновлённого дерева.

Класс `cl_1`:

- функционал:

- о метод `cl_1` — конструирует объект производного класса с указателем на головной объект и наименованием путём вызова конструктора базового класса.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	cl_base			Базовый класс для всех объектов системы, обеспечивающий возможность построения иерархического дерева объектов, где каждый объект может иметь головной объект и множество подчинённых объектов	
		cl_application	public		2
		cl_1	public		3
2	cl_application			Класс объекта-приложения, являющийся корневым в дереве иерархии. Обеспечивает построение дерева объектов согласно входным данным и запуск основного алгоритма	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
				(редактирование имён объектов по уровням и индексам)	
3	cl_1			Класс, производный от базового класса cl_base, используется для создания всех объектов дерева иерархии (кроме корневого). Демонстрирует механизм наследования от базового класса.	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса `cl_base`

Функционал: параметризованный конструктор для инициализации объекта.

Параметры: указатель `p_head_object` на объект базового класса `cl_base`, содержащий адрес головного объекта; строковый `s_object_name` - наименование создаваемого объекта (по умолчанию "Base_object").

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса `cl_base`

№	Предикат	Действия	№ перехода
1		присвоить свойству <code>p_head_object</code> текущего объекта значение входного параметра <code>p_head_object</code>	2
2		присвоить свойству <code>s_object_name</code> текущего объекта значение входного параметра <code>s_object_name</code>	3
3	есть ли головной объект?	вызов метода <code>push_back</code> у массива <code>subordinate_objects</code> по указателю <code>p_head_object</code> с указателем <code>this</code> , содержащим адрес текущего объекта, в качестве аргумента	4
			4
4			∅

3.2 Алгоритм деструктора класса `cl_base`

Функционал: деструктор для удаления текущего объекта и всех его подчинённых объектов.

Параметры: нет.

Алгоритм деструктора представлен в таблице 3.

Таблица 3 – Алгоритм деструктора класса `cl_base`

№	Предикат	Действия	№ перехода
1		инициировать целочисленную переменную цикла i нулевым значением	2
2	значение индекса i меньше размера вектора подчинённых объектов	удалить динамически созданный объект, адрес которого хранится в i -том элементе массива подчинённых объектов текущего объекта	3
			Ø
3		инкрементировать i	2

3.3 Алгоритм метода `set_name` класса `cl_base`

Функционал: редактирует имя объекта с проверкой отсутствия дуближа среди подчинённых объектов у головного.

Параметры: строк. `new_name` - новое наименование для текущего объекта.

Возвращаемое значение: логический тип `bool` - истина, если имя успешно изменено; ложь, если возник дуближ имени среди подчинённых головного объекта.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода `set_name` класса `cl_base`

№	Предикат	Действия	№ перехода
1	есть ли головной объект?		2

№	Предикат	Действия	№ перехода
			6
2		инициировать целочисленную переменную цикла i нулевым значением	3
3	значение индекса i меньше количества подчинённых объектов головного объекта		4
			6
4	имя текущего проверяемого объекта совпадает со значением входного параметра new_name	вернуть ложь	∅
			5
5		инкрементировать i	3
6		присвоить свойству s_object_name текущего объекта значение входного параметра new_name	7
7		вернуть истину	∅

3.4 Алгоритм метода get_name класса cl_base

Функционал: возвращает наименование объекта.

Параметры: нет.

Возвращаемое значение: строковая переменная, содержащая текущее наименование объекта.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода get_name класса cl_base

№	Предикат	Действия	№ перехода
1		вернуть значение строкового свойства s_object_name текущего объекта	∅

3.5 Алгоритм метода `get_head` класса `cl_base`

Функционал: возвращает адрес головного объекта текущего объекта.

Параметры: нет.

Возвращаемое значение: адрес головного объекта (возвращает `nullptr`, если текущий объект является корневым).

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода `get_head` класса `cl_base`

№	Предикат	Действия	№ перехода
1		вернуть значение указателя на головной объект <code>p_head_object</code> текущего объекта	Ø

3.6 Алгоритм метода `get_subordinate_count` класса `cl_base`

Функционал: возвращает количество подчинённых объектов текущего объекта.

Параметры: нет.

Возвращаемое значение: целочисленное значение, содержащее текущее количество подчинённых объектов.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода `get_subordinate_count` класса `cl_base`

№	Предикат	Действия	№ перехода
1		вернуть результат вызова метода <code>size()</code> для массива подчинённых объектов текущего объекта	Ø

3.7 Алгоритм метода `get_subobject_by_name` класса `cl_base`

Функционал: возвращает адрес подчинённого объекта по его имени (если не

найден - возвращает nullptr).

Параметры: строковая переменная name - наименование искомого подчинённого объекта.

Возвращаемое значение: адрес найденного подчинённого объекта (возвращает nullptr, если объект с заданным именем не найден).

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода *get_subobject_by_name* класса *cl_base*

№	Предикат	Действия	№ перехода
1		инициировать целочисленную переменную цикла i нулевым значением	2
2	значение индекса i меньше количества элементов в списке подчинённых объектов текущего объекта		3
		вернуть значение нулевого указателя	∅
3	имя текущего подчинённого объекта совпадает со значением входного параметра name	вернуть значение i-того указателя из массива подчинённых объектов	∅
		инкрементировать i	2

3.8 Алгоритм метода *get_subobject_by_index* класса *cl_base*

Функционал: возвращает адрес подчинённого объекта по его индексу (от 1 до количества подчинённых).

Параметры: целочисленная переменная index, содержащая порядковый номер подчинённого объекта (от 1 до количества подчинённых).

Возвращаемое значение: адрес найденного подчинённого объекта

(возвращает nullptr, если индекс выходит за допустимые пределы).

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода *get_subobject_by_index* класса *cl_base*

№	Предикат	Действия	№ перехода
1	значение параметра <i>index</i> меньше 1 или превышает текущее количество подчинённых объектов	вернуть значение нулевого указателя	Ø
		вернуть значение указателя на объект из списка подчинённых объектов по индексу <i>index</i> - 1	Ø

3.9 Алгоритм метода *print_tree* класса *cl_base*

Функционал: выводит наименования всех объектов дерева иерархии слева направо и сверху вниз.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода *print_tree* класса *cl_base*

№	Предикат	Действия	№ перехода
1	пуст ли список подчинённых объектов текущего объекта?		Ø
			2
2		вывод в консоль символа переноса строки и результата вызова метода <i>get_name()</i> по указателю на текущий объект	3
3		инициировать целочисленную переменную цикла <i>i</i> нулевым значением	4
4	значение индекса <i>i</i> меньше	вывод в консоль " " и результата вызова метода	5

№	Предикат	Действия	№ перехода
	количества подчинённых объектов текущего объекта	get_name() по указателю на i-тый элемент списка подчинённых объектов	
			∅
5		вызов метода print_tree() по указателю на i-тый элемент списка подчинённых объектов	6
6		инкрементировать i	4

3.10 Алгоритм конструктора класса cl_application

Функционал: создаёт корневой объект приложения, предварительно вызывая конструктор базового класса.

Параметры: указатель p_head_object на объект базового класса cl_base, содержащий адрес головного объекта; строковый s_object_name - наименование создаваемого объекта (по умолчанию "Application").

Алгоритм конструктора представлен в таблице 11.

Таблица 11 – Алгоритм конструктора класса cl_application

№	Предикат	Действия	№ перехода
1			∅

3.11 Алгоритм метода build_tree_objects класса cl_application

Функционал: строит исходное дерево иерархии объектов: чтение входных данных, создание объектов с проверкой уникальности имён и правил добавления.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 12.

Таблица 12 – Алгоритм метода *build_tree_objects* класса *cl_application*

№	Предикат	Действия	№ перехода
1		объявление строковых переменных s_head и s_sub	2
2		инициировать указатель p_head на объект класса cl_base адресом текущего объекта	3
3		инициировать указатель p_sub на объект класса cl_base нулевым значением (nullptr)	4
4		считать значение из стандартного потока ввода в переменную s_head	5
5		вызов метода set_name текущего объекта с параметром s_head	6
6		считать значение из стандартного потока ввода в переменные s_head и s_sub	7
7	значения переменных s_head и s_sub совпадают		∅
			8
8	значение указателя p_sub не нулевое и значение метода get_name() по указателю p_sub совпадает со значением s_head	присвоить указателю p_head значение p_sub	9
			9
9	в списке подчинённых объектов p_head отсутствует объект с именем s_sub и имя текущего объекта p_head совпадает с s_head	динамическое создание объекта p_sub класса cl_1 с аргументами p_head и s_sub и присвоение его адреса указателю p_sub	6
			6

3.12 Алгоритм метода `exec_app` класса `cl_application`

Функционал: запускает систему: чтение команд редактирования имён объектов по уровням и индексам, изменение имён и вывод обновлённого дерева.

Параметры: нет.

Возвращаемое значение: целое - код возврата программы, определяющий корректность выполнения работы.

Алгоритм метода представлен в таблице 13.

Таблица 13 – Алгоритм метода `exec_app` класса `cl_application`

№	Предикат	Действия	№ перехода
1		вывод на консоль результата вызова метода <code>get_name()</code> по указателю на текущий объект	2
2		вызов метода <code>print_tree()</code> по указателю на текущий объект	3
3		объявление целочисленных переменных <code>level</code> и <code>index</code>	4
4		объявление строковой переменной <code>new_name</code>	5
5	можно ли ввести <code>level</code> и <code>index</code> и <code>index</code> не равен 0?	считать значение из стандартного потока ввода в переменную <code>new_name</code>	6
			15
6		инициировать указатель <code>target</code> на объект класса <code>cl_base</code> адресом текущего объекта	7
7	значение переменной <code>index</code> больше единицы	инициировать целочисленную переменную цикла <code>i</code> нулевым значением	8
			14
8	значение индекса <code>i</code> меньше <code>level - 2</code>		9
			13

№	Предикат	Действия	№ перехода
9	значение указателя target нулевое		13
			10
10		присвоить target результат вызова метода get_subobject_by_index по указателю target с аргументом в виде результата вызова метода get_subordinate_count() по указателю target	11
11	значение указателя target нулевое		13
			12
12		инкрементировать i	8
13	значение указателя target не нулевое	присвоить target результат вызова метода get_subobject_by_index по указателю target с аргументом index	14
			14
14	значение указателя target не нулевое	вызов метода set_name по указателю на объект target с аргументом new_name	5
			5
15		вывод на консоль символа переноса строки и результата вызова метода get_name() по указателю на текущий объект	16
16		вызов метода print_tree() по указателю на текущий объект	17
17		вернуть 0	∅

3.13 Алгоритм функции main

Функционал: для организации работы программы: создания корневого объекта приложения, вызова методов построения дерева и запуска системы.

Параметры: нет.

Возвращаемое значение: целое - код возврата программы, определяющий корректность выполнения работы.

Алгоритм функции представлен в таблице 14.

Таблица 14 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		создание объекта <i>ob_cl_application</i> класса <i>cl_application</i> с аргументом <i>nullptr</i>	2
2		вызов метода <i>build_tree_objects</i> для объекта <i>ob_cl_application</i>	3
3		вернуть результат вызова метода <i>exes_app</i> для объекта <i>ob_cl_application</i>	Ø

3.14 Алгоритм конструктора класса *cl_1*

Функционал: конструирует объект производного класса с указателем на головной объект и наименованием путём вызова конструктора базового класса.

Параметры: указатель *p_head_object* на объект базового класса *cl_base*, содержащий адрес головного объекта; строковый *s_object_name* - наименование создаваемого объекта (по умолчанию "Base_object").

Алгоритм конструктора представлен в таблице 15.

Таблица 15 – Алгоритм конструктора класса *cl_1*

№	Предикат	Действия	№ перехода
1			Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-13.

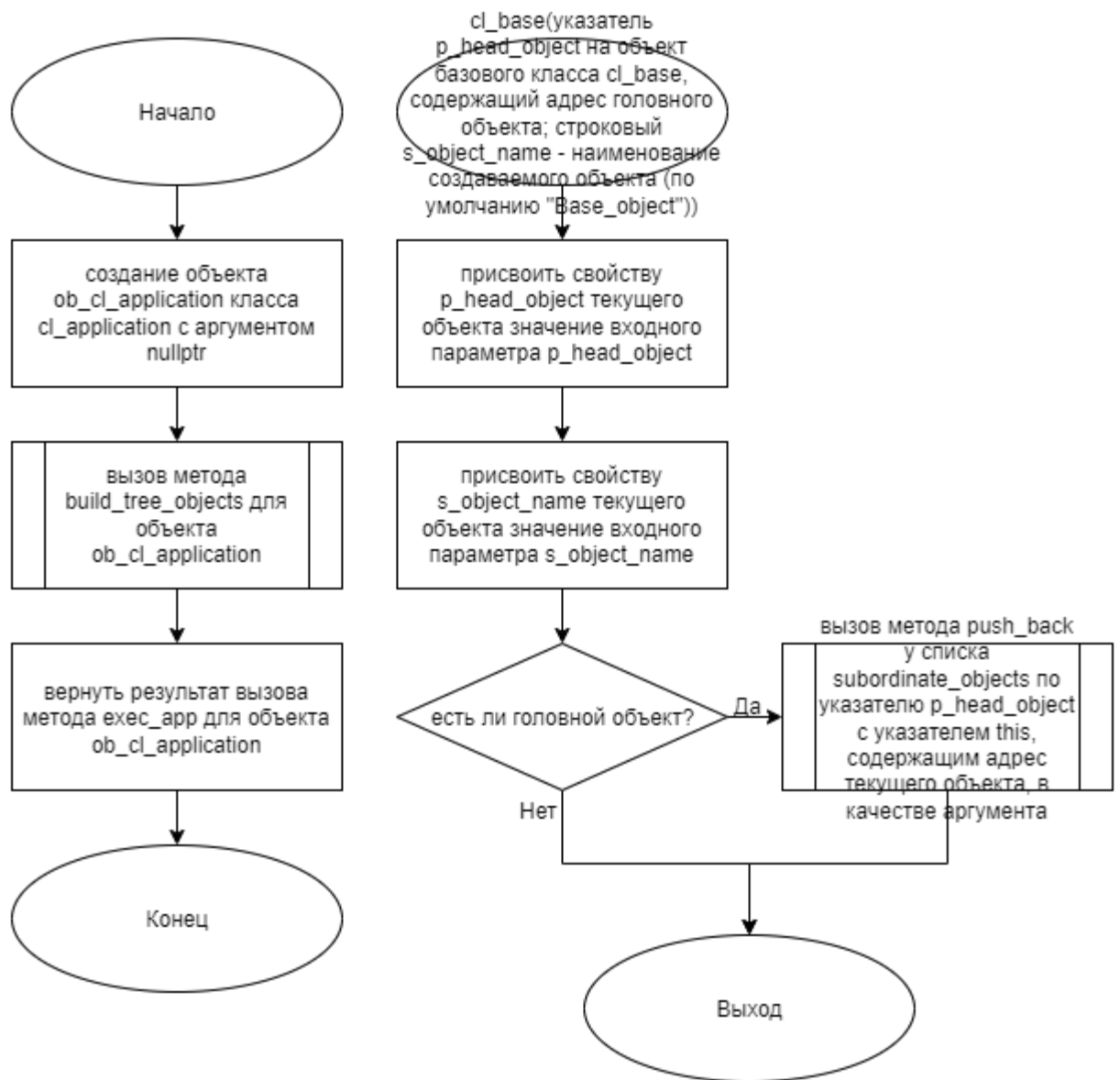


Рисунок 1 – Блок-схема алгоритма

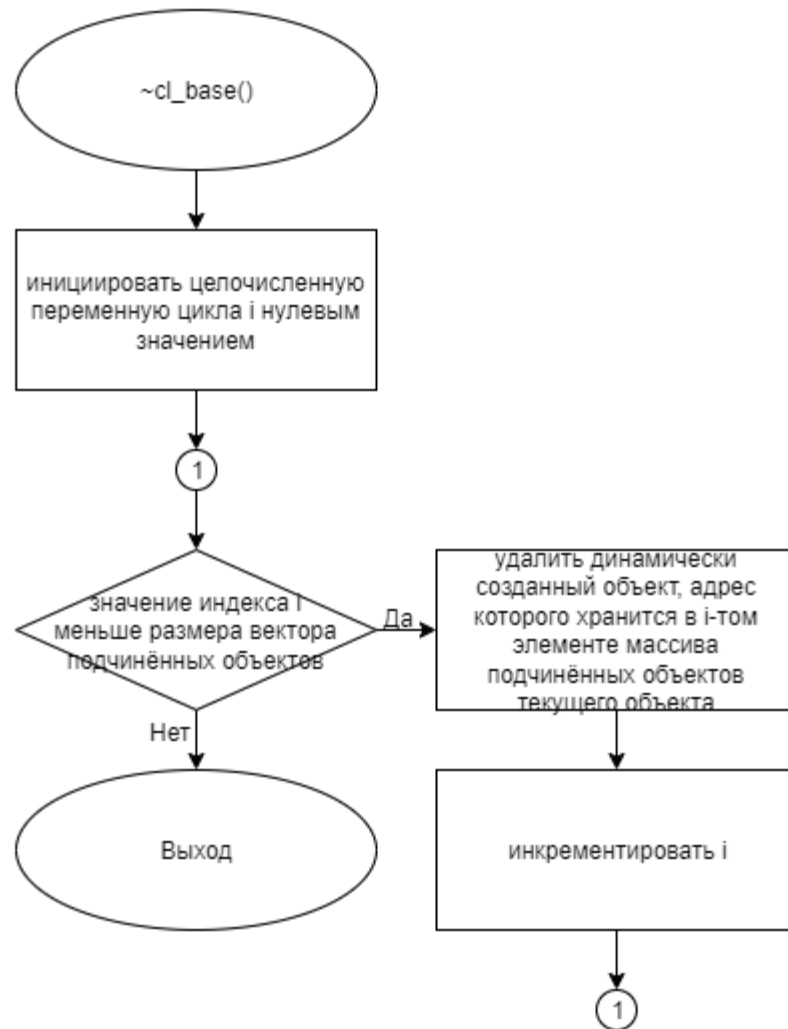


Рисунок 2 – Блок-схема алгоритма



Рисунок 3 – Блок-схема алгоритма

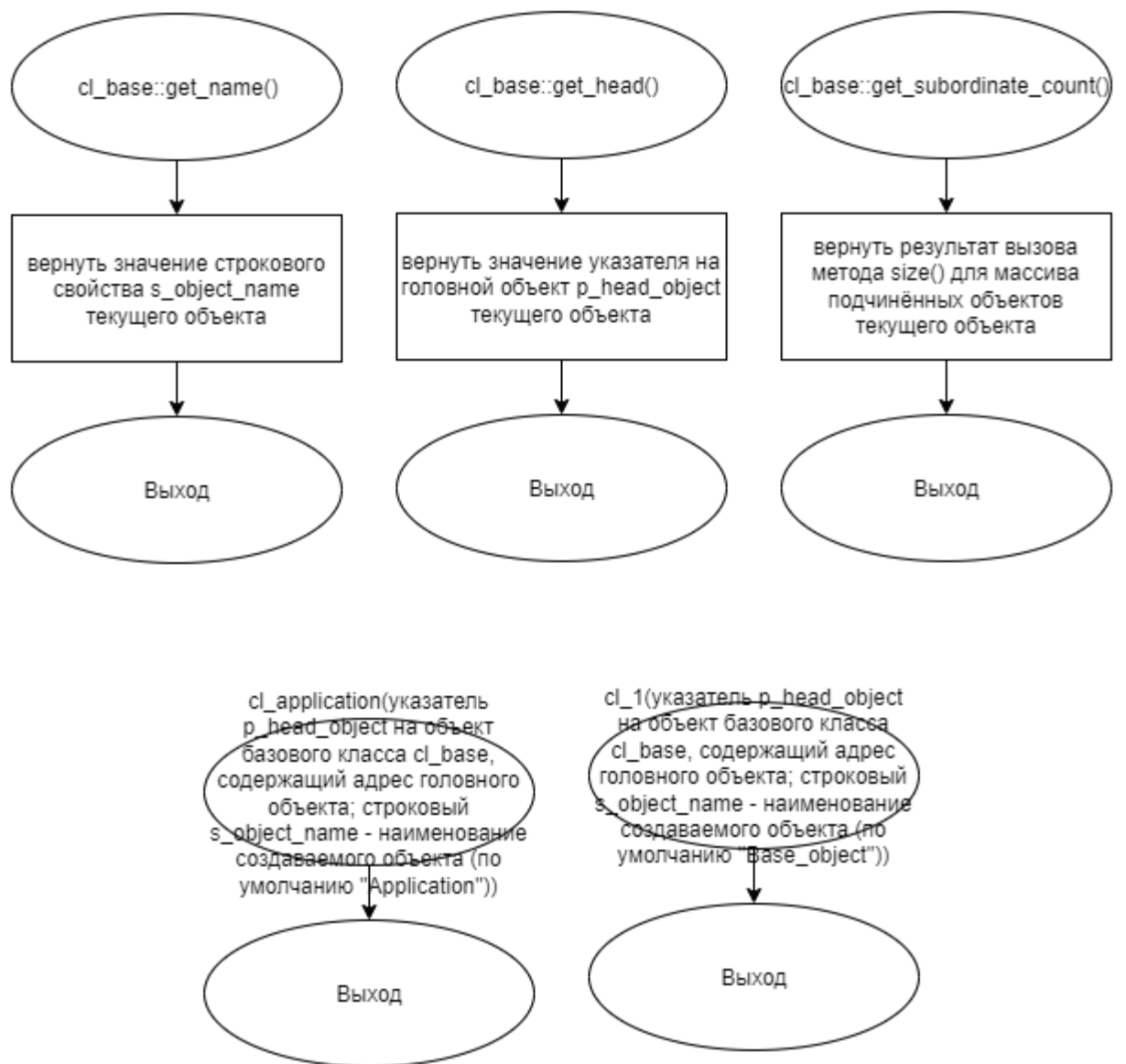


Рисунок 4 – Блок-схема алгоритма

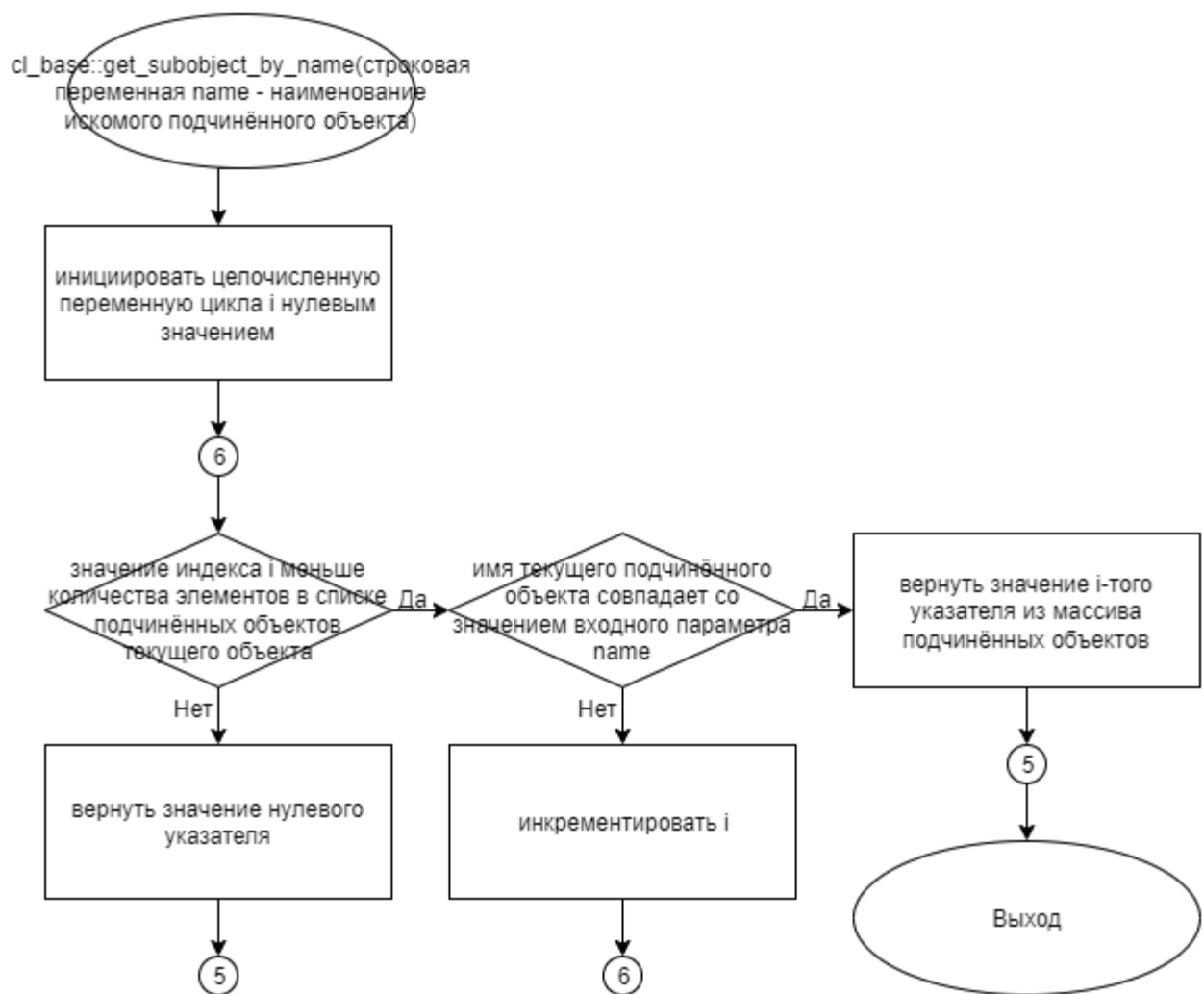


Рисунок 5 – Блок-схема алгоритма

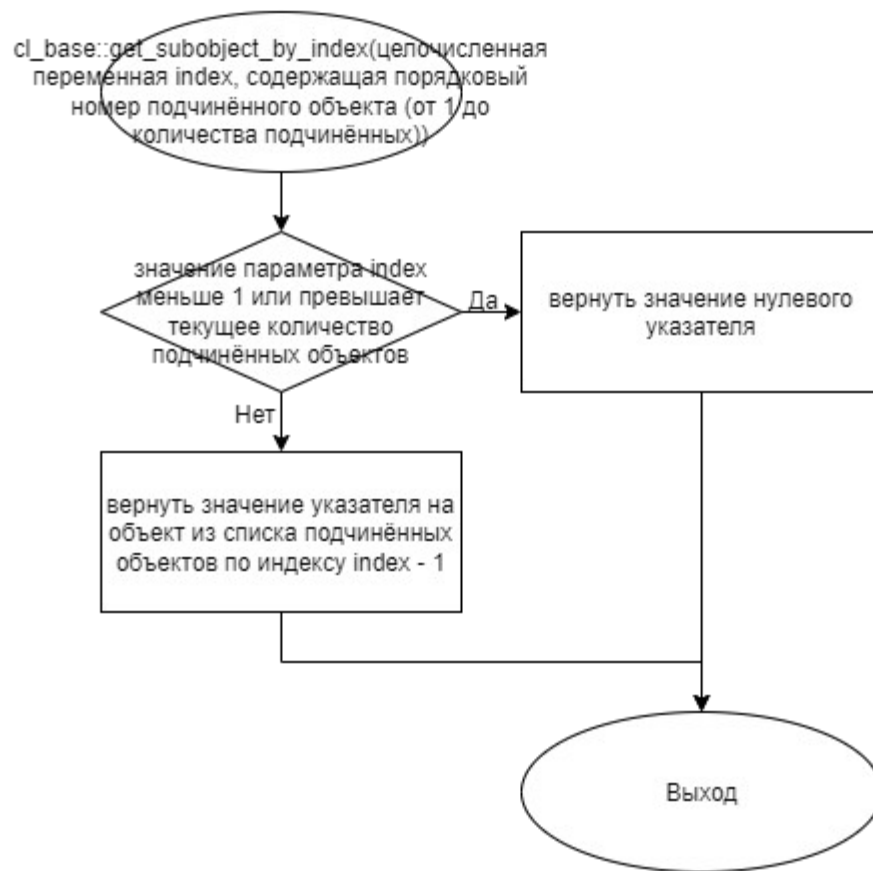


Рисунок 6 – Блок-схема алгоритма

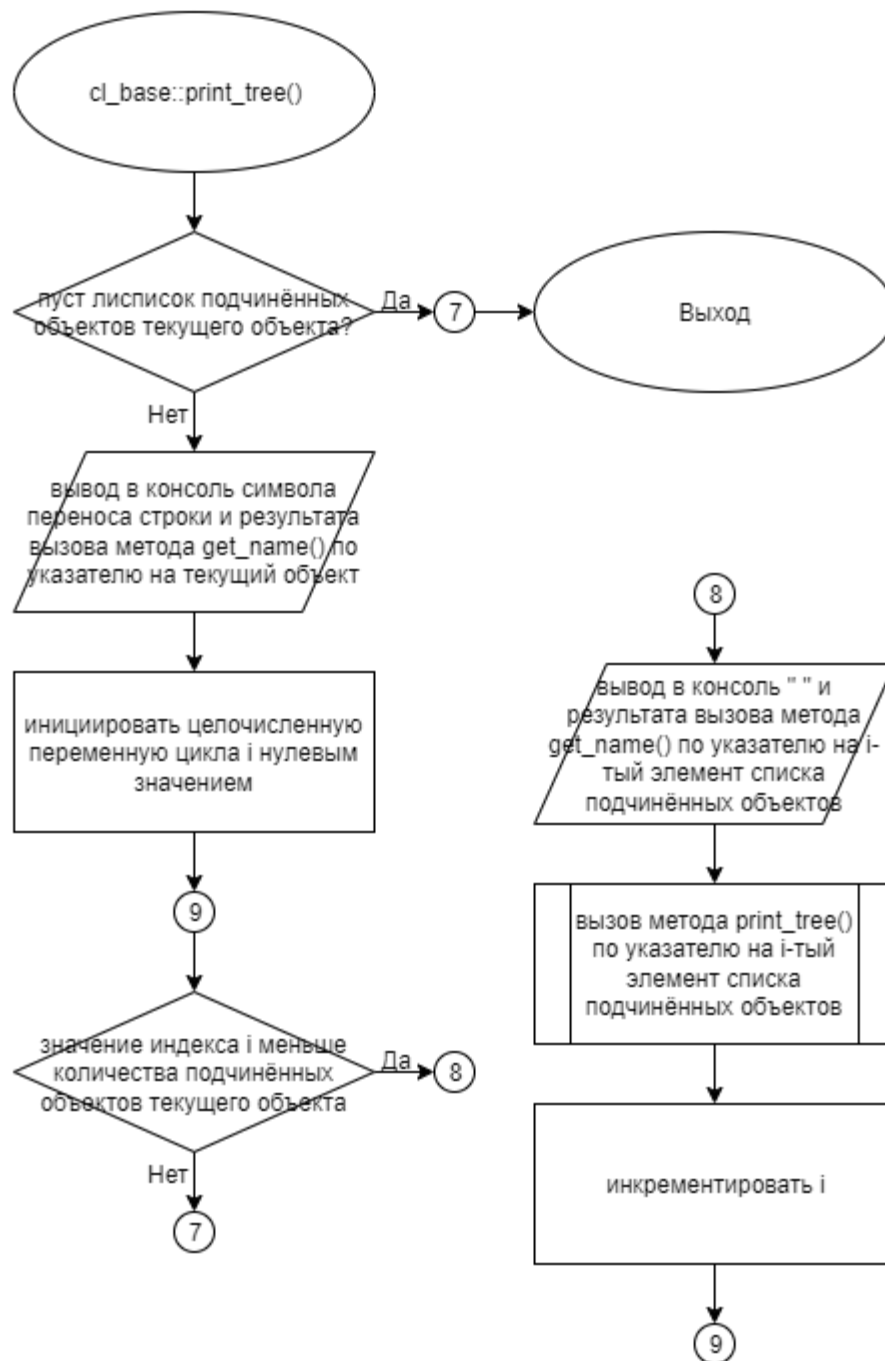


Рисунок 7 – Блок-схема алгоритма

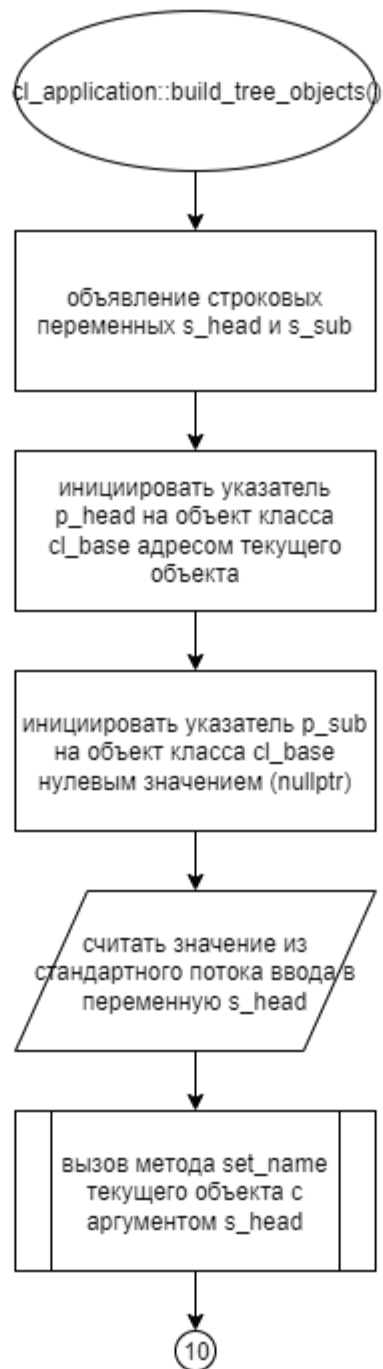


Рисунок 8 – Блок-схема алгоритма

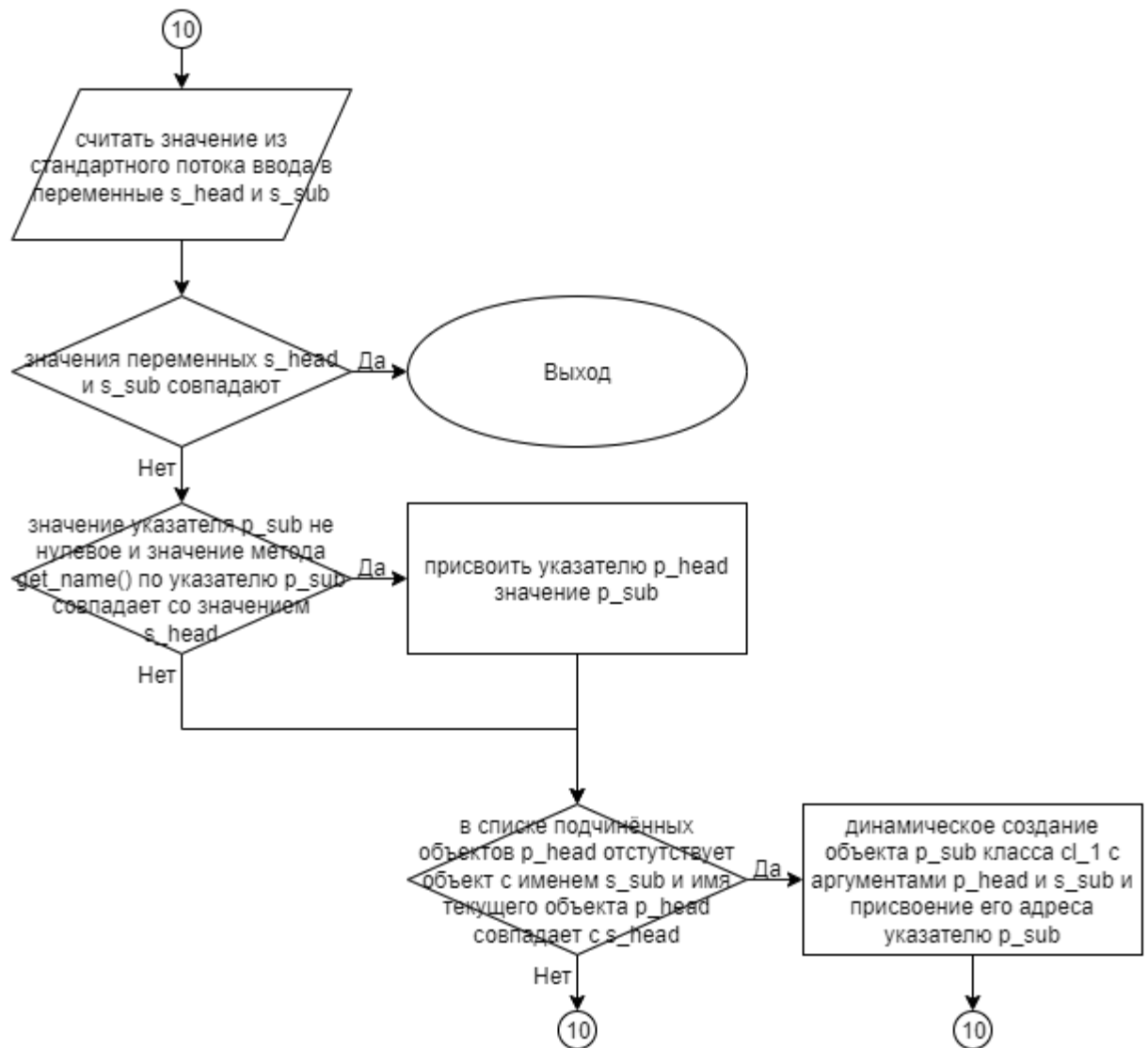


Рисунок 9 – Блок-схема алгоритма



Рисунок 10 – Блок-схема алгоритма

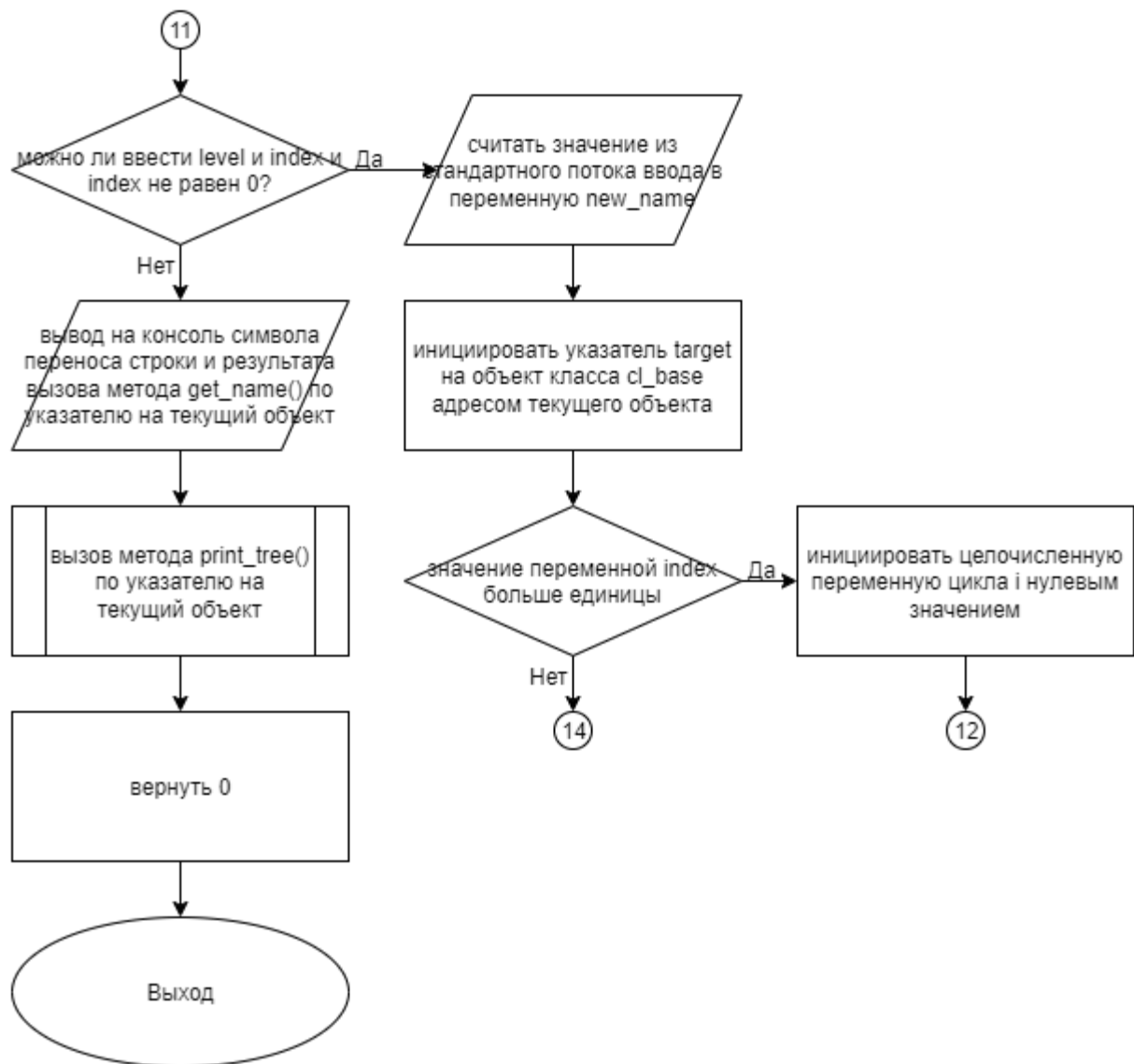


Рисунок 11 – Блок-схема алгоритма

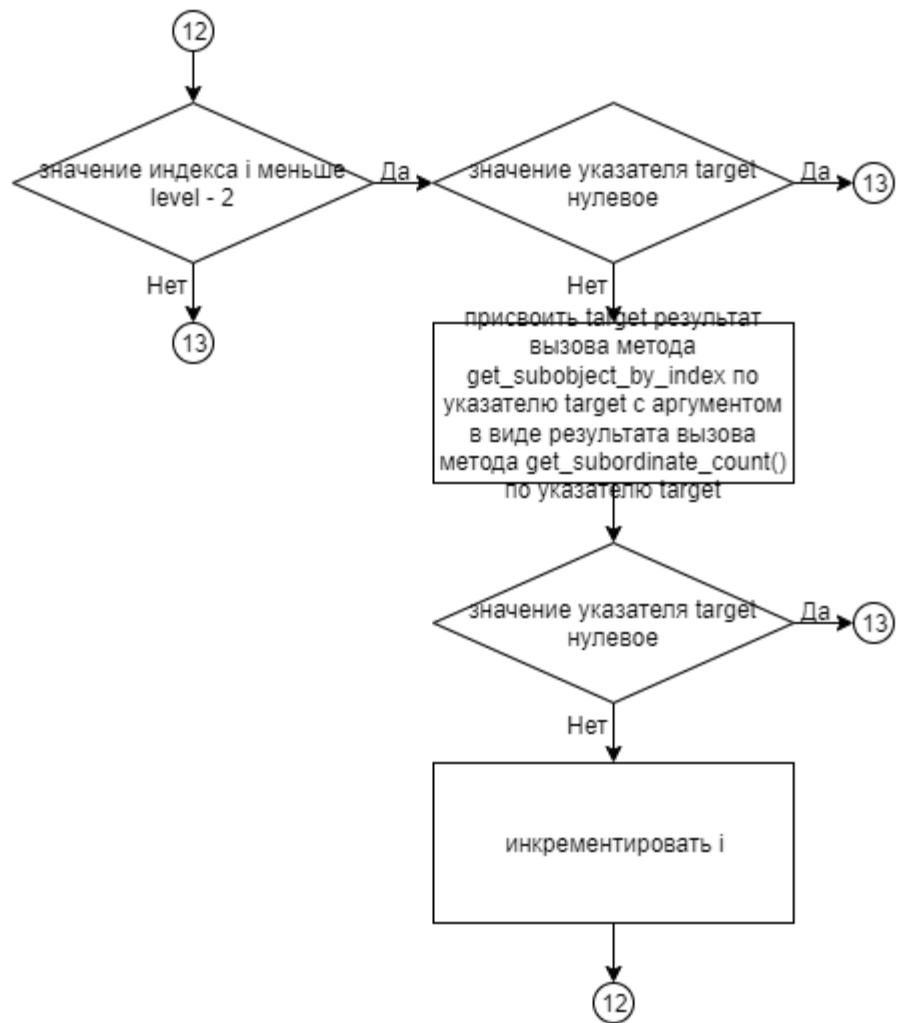


Рисунок 12 – Блок-схема алгоритма

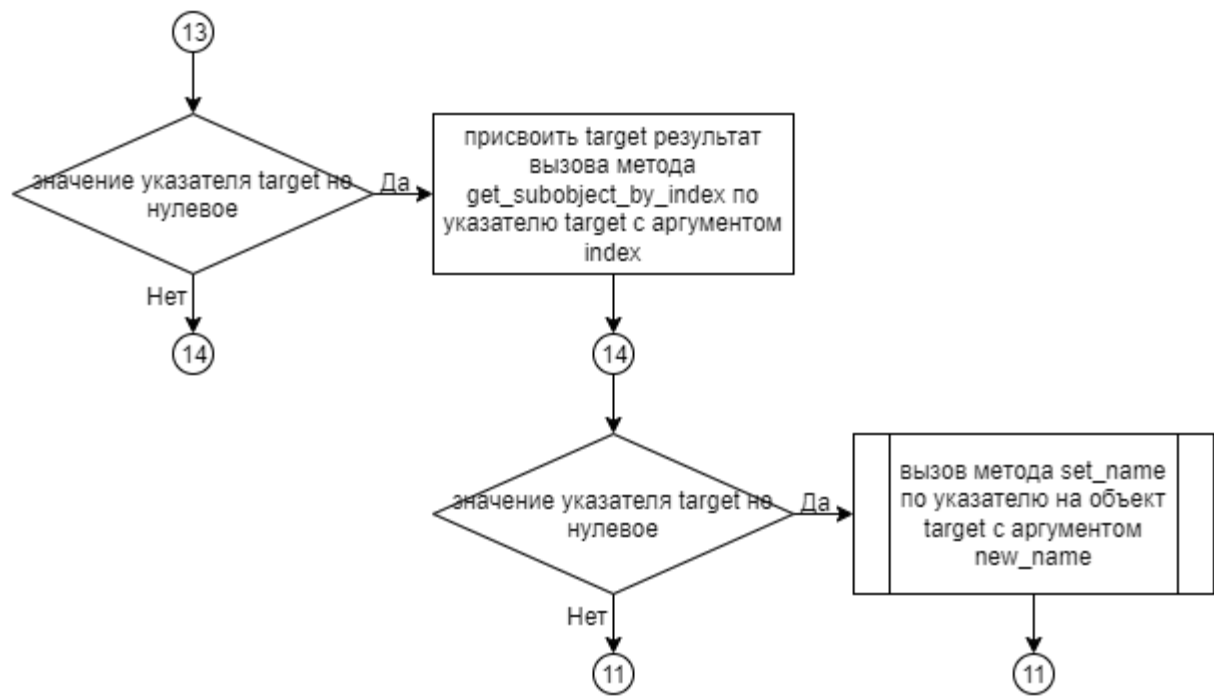


Рисунок 13 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл cl_1.cpp

Листинг 1 – cl_1.cpp

```
#include "cl_1.h"

cl_1 :: cl_1 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { } // конструирует объект
производного класса с указателем на головной объект и наименованием путём
вызова конструктора базового класса
```

5.2 Файл cl_1.h

Листинг 2 – cl_1.h

```
#ifndef __CL_1__H
#define __CL_1__H

#include "cl_base.h"

class cl_1 : public cl_base
{
public:
    cl_1 ( cl_base * p_head_object, string s_object_name ); //
параметризированный конструктор
};

#endif
```

5.3 Файл cl_application.cpp

Листинг 3 – cl_application.cpp

```
#include "cl_application.h"
```

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

cl_application :: cl_application ( cl_base * p_head_object, string
s_object_name )
    : cl_base ( p_head_object, s_object_name ) { } // создаёт корневой объект
приложения, предварительно вызывая конструктор базового класса

void cl_application :: build_tree_objects ( )
{
    string s_head, s_sub; // имена головного и
подчинённого объектов
    cl_base * p_head = this; // инициировать указатель
p_head на объект класса cl_base адресом текущего объекта
    cl_base * p_sub = nullptr; // инициировать указатель
p_sub на объект класса cl_base нулевым значением (nullptr)

    cin >> s_head;
    set_name ( s_head ); // устанавливаем имя
текущего объекта значением s_head вызовом метода set_name

    while ( true )
    {
        cin >> s_head >> s_sub;

        if ( s_head == s_sub )
        {
            break; // выход из цикла, если
значения переменных имена головного и подчинённого объектов совпадают
        }
        if ( p_sub != nullptr && s_head == p_sub -> get_name ( ) )
        {
            p_head = p_sub; // указатель p_head
перемещаем на последний созданный объект
        }
        if ( p_head -> get_subobject_by_name ( s_sub ) == nullptr && s_head ==
p_head -> get_name ( ) )
        {
            p_sub = new cl_1 ( p_head, s_sub ); // подчинённого с таким
именем ещё не существует и имя совпадает с именем головного, поэтому создаём
его
        }
    }
}

int cl_application :: exec_app ( ) // запускает систему:
чтение команд редактирования имён объектов по уровням и индексам, изменение
имён и вывод обновлённого дерева
{
    cout << this -> get_name ( ); // вывод имени текущего
объекта
    this -> print_tree(); // отображение иерархии
}

```

```

    int level, index;                                // объявление
целочисленных переменных level и index
    string new_name;

    while ( cin >> level >> index && level != 0 ) // можно ввести level и
index и index не равен 0?
    {
        cin >> new_name;
        cl_base * target = this;                    // присвоить target
результат вызова метода get_subobject_by_index по указателю target с
аргументом в виде результата вызова метода get_subordinate_count() по
указателю target

        if ( level > 1 )                             // значение переменной index
больше единицы
        {
            for ( int i = 0; i < level - 2; i++ ) // значение индекса i меньше
level - 2
            {
                if ( target == nullptr )             // значение указателя target
нулевое
                {
                    break;                            // прервать выполнение
внутреннего цикла
                }
                target = target -> get_subobject_by_index ( target ->
get_subordinate_count ( ) );
                if ( !target )                         // значение указателя target
нулевое
                {
                    break;                            // прервать выполнение
внутреннего цикла
                }
            }

            if ( target != nullptr )                 // значение указателя target
не нулевое
            {
                target = target -> get_subobject_by_index ( index );
            }
        }

        if ( target != nullptr )                     // значение указателя target
не нулевое
        {
            target -> set_name ( new_name );
        }
    }

    cout << endl << this -> get_name ( );
    this -> print_tree ( );                          // отображение итоговой
иерархии

    return 0;

```

```
}
```

5.4 Файл cl_application.h

Листинг 4 – cl_application.h

```
#ifndef __CL_APPLICATION__H
#define __CL_APPLICATION__H

#include "cl_base.h"
#include "cl_1.h"

class cl_application : public cl_base
{
public:
    cl_application ( cl_base * p_head_object, std :: string s_object_name =
"Application" ); // параметризованный конструктор
    void build_tree_objects ( );
    // строит исходное дерево иерархии объектов: чтение входных данных, создание
    // объектов с проверкой уникальности имён и правил добавления
    int exes_app ( );
    // запускает систему: чтение команд редактирования имён объектов по уровням
    // и индексам, изменение имён и вывод обновлённого дерева
};

#endif
```

5.5 Файл cl_base.cpp

Листинг 5 – cl_base.cpp

```
#include "cl_base.h"
#include <iostream>
using namespace std;

cl_base :: cl_base ( cl_base * p_head_object, string s_object_name ) //
параметризованный конструктор
{
    this -> s_object_name = s_object_name; //
    хранение наименования объекта
    this -> p_head_object = p_head_object; //
    хранение адреса на головной объект

    if ( p_head_object ) { // есть
    ли головной объект?
        p_head_object -> subordinate_objects.push_back ( this ); //
```

```

добавляем адрес текущего объекта в список адресов подчинённых объектов
головного объекта
    }
}

cl_base :: ~cl_base ( ) //
деструктор
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ ) //
    {
        значение индекса i меньше размера вектора подчинённых объектов?
        delete subordinate_objects [ i ]; // удалить
        динамически созданный объект, адрес которого хранится в i-том элементе
        массива подчинённых объектов текущего объекта
    }
}

bool cl_base :: set_name ( string new_name ) //
редактирует имя объекта с проверкой отсутствия дуближа среди подчинённых
объектов у головного
{
    if ( this -> p_head_object != nullptr ) // есть
    ли головной объект?
    {
        for ( int i = 0; i < p_head_object -> get_subordinate_count ( ); i++ ) //
        {
            значение индекса i меньше количества подчинённых объектов головного объекта
            if ( p_head_object -> subordinate_objects [ i ] -> get_name ( ) ==
            new_name ) //
            {
                проверяем уникальность имени
                return false; // есть
                другой подчинённый с таким же именем
            }
        }
    }

    this -> s_object_name = new_name; //
установка нового имени
    return true; // имя
успешно изменено
}

string cl_base :: get_name ( ) //
возвращает наименование объекта
{
    return this -> s_object_name; //
    текущее наименование объекта
}

cl_base * cl_base :: get_head() //
возвращает адрес головного объекта текущего объекта
{
    return this -> p_head_object; // адрес
    головного объекта текущего объекта
}

```

```

}

int cl_base :: get_subordinate_count ( ) //
возвращает количество подчинённых объектов текущего объекта
{
    return (int) this -> subordinate_objects.size ( ); //
    текущее количество подчинённых объектов
}

cl_base * cl_base :: get_subobject_by_name ( string name ) //
возвращает адрес подчинённого объекта по его имени (если не найден -
возвращает nullptr)
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ ) //
    значение индекса i меньше количества элементов в списке подчинённых объектов
    текущего объекта
    {
        if ( subordinate_objects [ i ] -> s_object_name == name ) // имя
        { // имя
            return subordinate_objects [ i ]; // вернуть
            значение i-того указателя из массива подчинённых объектов
        }
    }

    return nullptr; //
    объект не найден
}

cl_base * cl_base :: get_subobject_by_index ( int index ) //
возвращает адрес подчинённого объекта по его индексу (от 1 до количества
подчинённых)
{
    if ( index < 1 || index > this -> get_subordinate_count ( ) ) //
    проверка на корректность индекса
    {
        return nullptr; // объект
        не найден
    }

    return subordinate_objects [ index - 1 ]; // такой
    подчинённый существует
}

void cl_base :: print_tree ( ) //
выводит наименования всех объектов дерева иерархии слева направо и сверху
вниз
{
    if ( this -> get_subordinate_count ( ) == 0 ) // пуст
    ли список подчинённых объектов текущего объекта?
    {
        return; //
        завершаем выполнение метода, если список подчинённых объектов текущего
        объекта пуст
    }
}

```

```

        cout << endl << this -> get_name ( );
        for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
        {
            cout << " " << subordinate_objects [ i ] -> get_name ( );
            subordinate_objects [ i ] -> print_tree ( );           // вызов
// метода print_tree() по указателю на i-тый элемент списка подчинённых
// объектов
        }
    }
}

```

5.6 Файл cl_base.h

Листинг 6 – cl_base.h

```

#ifndef __CL_BASE__H
#define __CL_BASE__H

#include <string>
#include <vector>
using namespace std;

class cl_base
{
protected:
    string s_object_name;
    // хранение наименования объекта
    cl_base * p_head_object;
    // хранение адреса на головной объект
    vector < cl_base * > subordinate_objects;
    // хранение динамического массива адресов подчинённых объектов

public:
    cl_base ( cl_base* p_head_object, string s_object_name = "Base_object" );
    // параметризованный конструктор
    ~cl_base ( );
    // деструктор для удаления текущего объекта и всех его подчинённых объектов

    bool set_name ( string new_name );
    // редактирует имя объекта с проверкой отсутствия дубликата среди подчинённых
    // объектов у головного
    string get_name ( );
    // возвращает наименование объекта
    cl_base * get_head ( );
    // возвращает адрес головного объекта текущего объекта
    int get_subordinate_count ( );
    // возвращает количество подчинённых объектов текущего объекта
    cl_base * get_subobject_by_name ( string name );
    // возвращает адрес подчинённого объекта по его имени (если не найден -
    // возвращает nullptr)
    cl_base * get_subobject_by_index ( int index );

```

```

// возвращает адрес подчинённого объекта по его индексу (от 1 до количества
подчинённых)
void print_tree ( );
// выводит наименования всех объектов дерева иерархии слева направо и сверху
вниз
};

#endif

```

5.7 Файл main.cpp

Листинг 7 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>

#include "cl_application.h"
using namespace std;

int main()
{
    cl_application ob_cl_application ( nullptr ); // создание корневого
объекта
    ob_cl_application.build_tree_objects ( );      // конструирование системы,
построение дерева объектов
    return ob_cl_application.exec_app ( );        // запуск системы
}

```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 16.

Таблица 16 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Object_root Object_root Object_1 Object_root Object_2 Object_root Object_3 Object_3 Object_4 Object_3 Object_5 Object_6 Object_6 1 1 root_Object 3 1 Object_15 4 1 Object_41 2 7 Object_72 2 2 Object_55 0	Object_root Object_root Object_1 Object_2 Object_3 Object_3 Object_4 Object_5 root_Object root_Object Object_1 Object_55 Object_3 Object_3 Object_15 Object_5	Object_root Object_root Object_1 Object_2 Object_3 Object_3 Object_4 Object_5 root_Object root_Object Object_1 Object_55 Object_3 Object_3 Object_15 Object_5
Object_root Object_root Object_1 Object_root Object_2 Object_2 Object_3 Object_2 Object_4 Object_3 Object_3 1 1 root_Object 3 1 Object_67 2 2 Object_52 0	Object_root Object_root Object_1 Object_2 Object_2 Object_3 Object_4 root_Object root_Object Object_1 Object_52 Object_52 Object_67 Object_4	Object_root Object_root Object_1 Object_2 Object_2 Object_3 Object_4 root_Object root_Object Object_1 Object_52 Object_52 Object_67 Object_4

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).